ЛАБОРАТОРНА РОБОТА No 3

Тема: Одновимірні та багатовимірні масиви, обробка символьних рядків у мові С та робота з гілками в Git

Мета роботи:

Набути практичних навичок обробки послідовних областей оперативної пам'яті: проектування алгоритмів фільтрації, пошуку екстремумів та перестановок елементів в одновимірних масивах. Опанувати кеш-дружні алгоритми обробки двовимірних матриць з урахуванням апаратного стандарту розташування даних за рядками (Row-Major Order). Засвоїти низькорівневу специфіку роботи з текстовою інформацією через символьні масиви типу char із кінцевим нуль-термінатором (\0). Опанувати командну модель розгалуження в системі контролю версій Git: створення функціональних гілок (git branch , git checkout / git switch), фіксація змін та їхнє безпечне злиття в основну гілку (git merge). Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

Завдання 1. Обробка одновимірного масиву

Для виконання завдання було створено програму task1.c, яка формує масив із 15 цілих псевдовипадкових чисел у діапазоні від −50 до 50, виводить його на екран і виконує повний реверс елементів.

Реверс здійснюється шляхом обміну першого елемента з останнім, другого — з передостаннім і так далі. Для обміну використовується тимчасова змінна temp. Допоміжний масив не створюється.

Лістинг програми task1.c

```c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 15

int main(void) {
    int arr[SIZE];
    int i, temp;

    srand((unsigned int)time(NULL));

    for (i = 0; i < SIZE; i++) {
        arr[i] = rand() % 101 - 50;
    }

    printf("Original array:\n");
    for (i = 0; i < SIZE; i++) {
        printf("%4d", arr[i]);
    }
    printf("\n");

    for (i = 0; i < SIZE / 2; i++) {
        temp = arr[i];
        arr[i] = arr[SIZE - 1 - i];
        arr[SIZE - 1 - i] = temp;
    }

    printf("Reversed array:\n");
    for (i = 0; i < SIZE; i++) {
        printf("%4d", arr[i]);
    }
    printf("\n");

    return 0;
}
```

```
PS C:\Users\Ivanu\Projects\programming-semI\lab03> .\task1.exe
Original array:
  40 -36   8  34  12  39  15 -38  16   9 -37  41 -31 -10 -49
Reversed array:
 -49 -10 -31  41 -37   9  16 -38  15  39  12  34   8 -36  40
```

Завдання 2. Обробка двовимірної матриці

Для виконання завдання було створено програму task2.c, яка формує квадратну матрицю розміром 5 × 5 та заповнює її псевдовипадковими цілими числами в діапазоні від −50 до 50.Після виведення початкової матриці програма перевіряє умови належності елемента до головної та побічної діагоналей. Для головної діагоналі виконується умова i == j, а для побічної — i + j == SIZE - 1.Елемент, який одночасно належить обом діагоналям, розташований у центрі матриці. Для матриці непарного розміру 5 × 5 це елемент з індексами [2][2]. Програма визначає його значення та виводить результат.

Лістинг програми task2.c

```c

#define SIZE 5

int main(void) {
    int matrix[SIZE][SIZE];
    int i, j;
    int max_element = 0;
    int found = 0;

    srand((unsigned int)time(NULL));

    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            matrix[i][j] = rand() % 101 - 50;
        }
    }

    printf("Original matrix:\n");

    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            printf("%6d", matrix[i][j]);
        }
        printf("\n");
    }

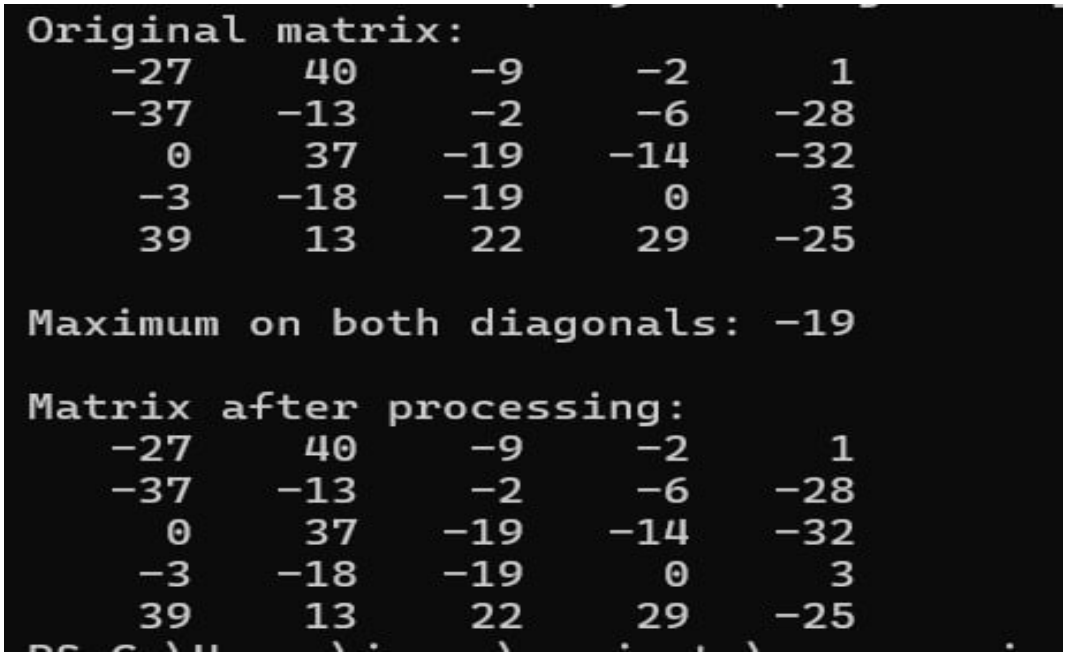
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            if (i == j && i + j == SIZE - 1) {
                if (!found || matrix[i][j] > max_element) {
                    max_element = matrix[i][j];
                    found = 1;
                }
            }
        }
    }

    if (found) {
        printf("\nMaximum on both diagonals: %d\n", max_element);
    } else {
        printf("\nNo common diagonal element found.\n");
    }

    printf("\nMatrix after processing:\n");

    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            printf("%6d", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```



Завдання 3. Обробка символьних рядків

Для виконання завдання було створено програму `task3.c`, яка зчитує рядок із клавіатури та перевіряє наявність символів, що повторюються три або більше разів поспіль.Для зберігання тексту використовується символьний масив розміром 256 байтів. Введення здійснюється за допомогою функції `fgets()`, яка обмежує кількість символів, що зчитуються.Обробка виконується вручну за допомогою циклу `while`. Змінна `count` підраховує кількість однакових символів поспіль, а змінна `found` зберігає результат перевірки. Для завершення обробки використовується нуль-термінатор `\0`.

Лістинг програми `task3.c`

```c
#define BUFFER_SIZE 256

int main(void) {
    char str[BUFFER_SIZE];
    int i = 0;
    int count = 1;
    int found = 0;

    printf("Enter a string: ");

    if (fgets(str, sizeof(str), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    while (str[i] != '\0' && str[i] != '\n') {
        i++;
    }

    str[i] = '\0';
    i = 0;

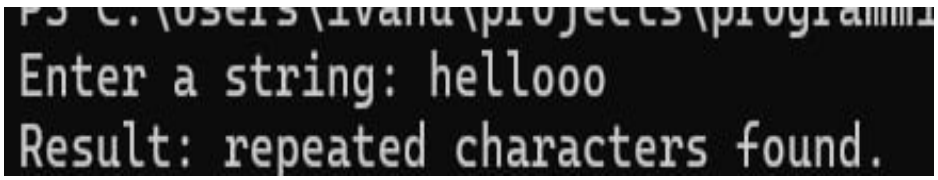
    while (str[i] != '\0') {
        if (str[i] == str[i + 1]) {
            count++;

            if (count >= 3) {
                found = 1;
            }
        } else {
            count = 1;
        }

        i++;
    }

    if (found) {
        printf("Result: repeated characters found.\n");
    } else {
        printf("Result: no repeated characters found.\n");
    }

    return 0;
}
```



ВИСНОВКИ:

Під час виконання лабораторної роботи було закріплено навички програмування мовою С та роботи з масивами різних типів.У першому завданні реалізовано реверс одновимірного масиву на місці без використання допоміжного масиву. У другому завданні створено квадратну матрицю та реалізовано пошук елемента, який одночасно належить головній і побічній діагоналям. У третьому завданні реалізовано посимвольний аналіз рядка для визначення повторення однакових символів три або більше разів поспіль.Також було опрацьовано використання циклів, умовних операторів, символьних масивів, нуль-термінатора та основних команд Git для керування версіями програмного коду.Отже, мети лабораторної роботи досягнуто, а практичні навички роботи з масивами, символьними рядками та системою контролю версій Git закріплено.